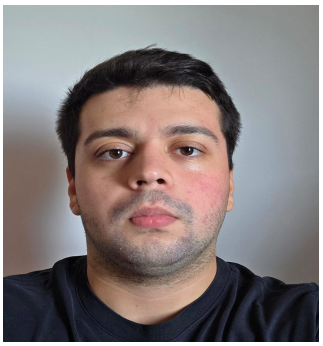
Lab 3 - The Metronome Thing with MQTT

Group: IOTPJ

VSS name: iotpj

Team Members

Member 1	Member 2	Member 3
Riccardo Carini	Nicola Ceccarini	Simone Pipola
		

Project Description

For this assignment, our group successfully developed the Metronome Thing with MQTT within our iotpj Virtual Smart Space (VSS). Extending our previous Lab 2 project, we transformed the communication infrastructure from a synchronous RESTful API into an event-driven, decoupled publish/subscribe IoT system powered by the Mosquitto MQTT broker.

To preserve real-time sensing and actuation precision, we maintained our hardware setup on the Raspberry Pi using two tactile pushbuttons (Mode Button on GPIO 21 and Tap Button on GPIO 26) as inputs and two LEDs (Green on GPIO 13 and Red on GPIO 16) as actuators. We adopted a hybrid architecture: a deterministic multi-threaded C++ service using wiringPi handles microsecond-accurate tempo tracking and LED control, interconnected via standard I/O IPC with an asynchronous Python client that manages MQTT communication with the broker.

The system logic is based on two operational modes:

- Play Mode: The program blinks the Green LED at the specified tempo (50% duty cycle) and keeps the Red LED off. On system startup, the tempo defaults to 0 BPM (Green LED remains off). Remote MQTT commands or local button presses dynamically adjust tempo in real time.
- Learn Mode: The program activates the Red LED upon each tap of the button to record tempo intervals in real time, requiring at least 4 taps to compute a valid BPM, while the Green LED remains off. Upon exiting Learn Mode, the system computes the average interval delta, updates the active BPM, and broadcasts the new state over MQTT.

The program handles sampling intervals and replaces the previous CRUD REST endpoints with a structured MQTT topic hierarchy (metronome/iotpj/). It publishes commands (cmd/set_bpm, cmd/set_mode, cmd/reset_min, cmd/reset_max) using Quality of Service 1 (QoS 1) to guarantee delivery. State topics (state/bpm, state/mode, state/min, state/max) use **retained messages** (retain = true) so new subscribers immediately receive the latest state without polling. In addition, the device configures a **Last Will and Testament (LWT)** on status, enabling the broker to broadcast an offline alert immediately if the device crashes or disconnects. Our remote interfaces include an interactive Web Dashboard (MQTT over WebSockets) and Python CLI tools.

Demo Video (YouTube): https://youtu.be/[YOUR_DEMO_VIDEO_LINK_HERE]

Distribution of Effort

The development of the Metronome Thing with MQTT was carried out through an entirely collaborative approach among all team members:

- All members coordinated to configure network parameters, set up and configure the Mosquitto broker (enabling both standard TCP port 1883 and WebSocket port 9001), and design the MQTT topic space.
- The hardware circuit on the breadboard was reused, and the team jointly implemented the hybrid architecture on the Raspberry Pi, interfacing the C++ metronome core (timing, 50ms debouncing, LED threads) with the Python MQTT bridge.
- Writing and refining the MQTT client and remote dashboard followed a collaborative programming methodology, integrating retained messages, QoS 1 handshakes, LWT offline alerts, and testing bidirectional synchronization.
- All members contributed equally to drafting the technical report, testing the prototype across multiple clients, and recording and editing the demonstration video.

Relative % of Effort

- Riccardo Carini: 33.3%
- Nicola Ceccarini: 33.4%
- Simone Pipola: 33.3%